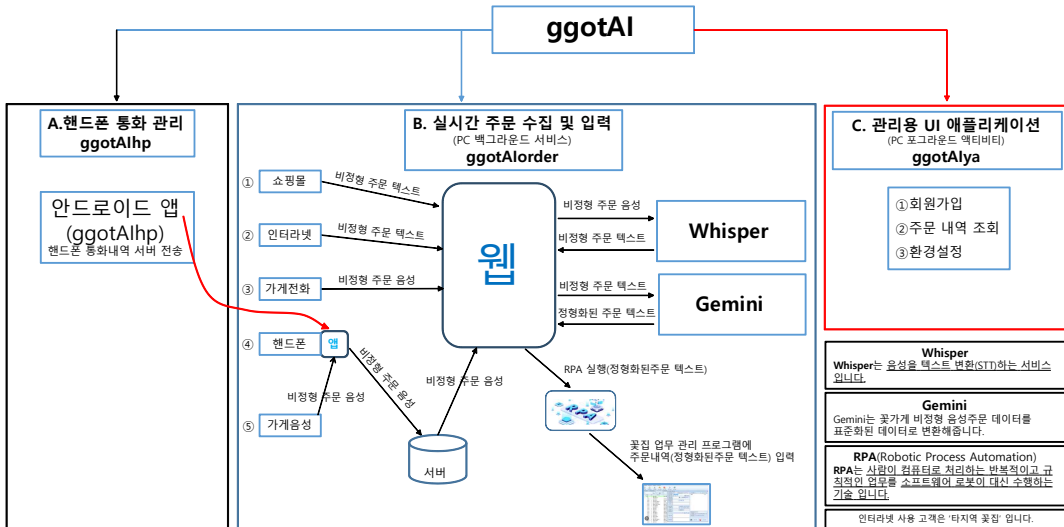
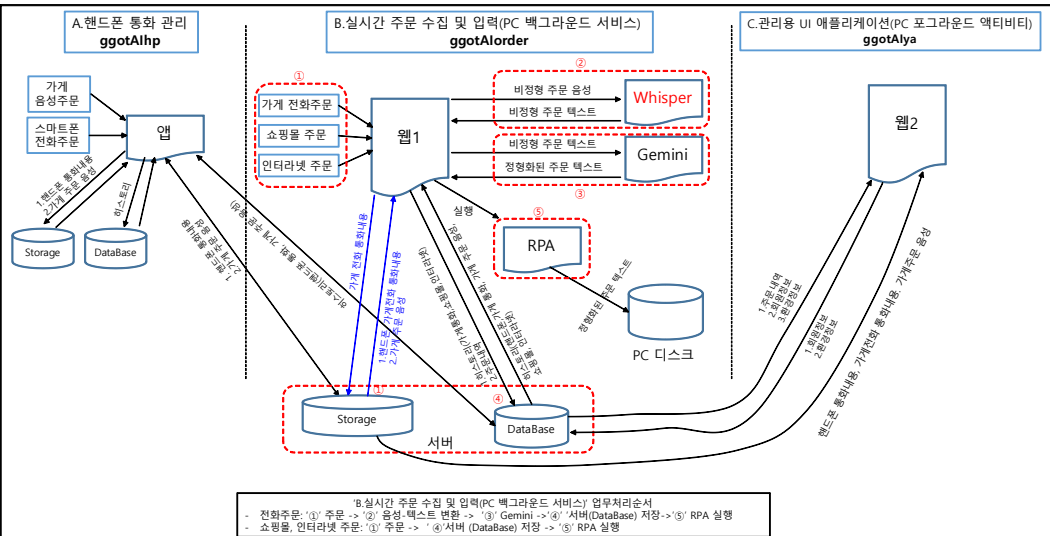


**I . ggotAI(꽃아이)**

## 1. ggotAI(꽃아이) 전체 업무 흐름



## 2. ggotAI(꽃아이) 시스템 구성도



### 3. DataBase & Storage

|          | Table 이름            | Table 간단 내용 | Table 저장 위치 | 내용                                                                                                     | 사용 SW<br>(R:Read, W:Write)                       |
|----------|---------------------|-------------|-------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| DataBase | server_call_history | 히스토리_서버     | 서버          | 각 주문채널(핸드폰, 가게전화, 가게음성, 쇼핑물, 인트라넷)로 부터 주문을 수집한 환경 정보(채널, 일자, 시간, 고객전화번호.... 등)                         | ggotAlhp(R,W)<br>ggotAlorder(R,W)<br>ggotAlya(R) |
|          | hp_call_history     | 히스토리_핸드폰    | 핸드폰         | - 핸드폰으로 부터 주문을 수집한 환경 정보(채널, 일자, 시간, 고객전화번호.... 등)<br>- 핸드폰에 저장하는 DataBase<br>- 주기적으로 히스토리(서버)와 동기화 필요 | ggotAlhp(R,W)                                    |
|          | Order_details       | 주문내역        | 서버          | 각 채널(핸드폰, 가게전화, 가게음성, 쇼핑물, 인트라넷)로 부터 수집된 주문내역                                                          | ggotAlorder(R,W)<br>ggotAlya(R)                  |
|          | Setting_info        | 환경정보        | 서버          | - ggotAl, ggotAlorder, ggotAlhp 사용을 위한 환경정보<br>- ggotAlhp 사용 핸드폰, 쇼핑물 주소...등                           | ggotAlhp(R)<br>ggotAlorder(R)<br>ggotAlya(R,W)   |
|          | member_info         | 회원정보        | 서버          | 사용자(꽃집) 정보                                                                                             | ggotAlhp(R)<br>ggotAlorder(R)<br>ggotAlya(R,W)   |

|         | 파일 이름 | 파일 간단 내용   | 저장 위치 | 내용                                                     | 사용 SW<br>(R:Read, W:Write)                     |
|---------|-------|------------|-------|--------------------------------------------------------|------------------------------------------------|
| Storage |       | 핸드폰 통화내용   | 서버    | 주문 통화 내용(핸드폰, 가게전화, 가게 음성)                             | ggotAlhp(R,W)<br>ggotAlorder(R)<br>ggotAlya(R) |
|         |       | 핸드폰 통화내용   | 핸드폰   | -핸드폰 주문 통화 내용(핸드폰에 저장)<br>-별도로 저장하지 않고 안드로이드 폰이 자동 저장함 | ggotAlhp(R,W)                                  |
|         |       | 가게 전화 통화내용 | 서버    | 가게 전화의 주문 통화 내용                                        | ggotAlorder(R,W)<br>ggotAlya(R)                |
|         |       | 가게 주문 음성   | 서버    | 가게의 음성 주문 내역(서버)                                       | ggotAlhp(R,W)<br>ggotAlorder(R)<br>ggotAlya(R) |
|         |       | 가게 주문 음성   | 핸드폰   | 가게의 음성 주문 내역_핸드폰에 저장                                   | ggotAlhp(R,W)                                  |

## 4. DataBase Table

CREATE TABLE server\_call\_history ( -- 기존 Supabase의 Server\_call\_history Table 수정해야 됨

```
id SERIAL PRIMARY KEY,
channel_order VARCHAR(20) DEFAULT '기타', -- 주문 수집 채널 ('핸드폰', '가게전화', '쇼핑몰', '인터넷', '가게음성', '기타')
channel_classification VARCHAR(255) NOT NULL, -- 채널별 상세 정보 (전화번호, 쇼핑몰 주소, 인터넷 주소 등)
shop_key INT NOT NULL, -- 꽃가게 고유 KEY (member_info 테이블의 id와 매핑)
shop_name VARCHAR(50) NOT NULL, -- 꽃가게 이름
customer_phone_number VARCHAR(20) DEFAULT '', -- 전화를 건 '고객'의 전화번호 (쇼핑몰/인터넷은 공백 문자)
customer_name VARCHAR(50) DEFAULT '신규', -- 고객명 (전화번호부 연동명, 없을 시 '신규' / 쇼핑몰/인터넷은 공백 문자)
call_date DATE NOT NULL, -- 통화/수집 날짜 (YYYY-MM-DD)
call_time TIME NOT NULL, -- 통화/수집 시간 (HH:mm:ss)
duration_seconds INT DEFAULT 0, -- 통화 시간(초 단위, 텍스트 채널은 0)
audio_file_name VARCHAR(255) DEFAULT NULL, -- 서버 스토리지에 저장된 음성 파일명 (텍스트 채널은 NULL)

-- 성능 최적화를 위해 파일명 대신 텍스트 원본을 DB에 직접 저장하도록 수정
stt_text TEXT DEFAULT NULL, -- Whisper가 변환한 비정형 주문 텍스트 원본 (쇼핑몰/인터넷은 스크래핑 원문)

is_order CHAR(1) DEFAULT 'N', -- Gemini(LLM)가 판별한 최종 주문 여부 ('Y' 또는 'N')
created_at TIMESTAMPTZ DEFAULT NOW(), -- 데이터 생성 일시
```

FOREIGN KEY (shop\_key) REFERENCES member\_info(id) ON DELETE CASCADE -- 회원 정보와 유기적 연동

);

CREATE TABLE order\_details ( -- 신규 생성해야 함

```
id SERIAL PRIMARY KEY,
call_history_id INT NOT NULL, -- 어떤 통화/채널에서 온 주문인지 매핑 (외래키)
shop_key INT NOT NULL, -- 꽃가게 고유 KEY [cite: 852]
shop_name VARCHAR(50) NOT NULL, -- 꽃가게 이름 [cite: 854]
customer_name VARCHAR(50) DEFAULT '신규', -- 고객명 [cite: 854]
customer_phone_number VARCHAR(20) NOT NULL, -- 고객전화번호 [cite: 855]
product_name VARCHAR(150) NOT NULL, -- 상품명 [cite: 856]
quantity INT DEFAULT 1, -- 수량 [cite: 857]
price INT DEFAULT 0, -- 가격 [cite: 858]
delivery_at TIMESTAMPTZ NOT NULL, -- 배달일시 [cite: 859]
delivery_place VARCHAR(255) NOT NULL, -- 배달장소 [cite: 860]
receiver_name VARCHAR(50) NOT NULL, -- 받는사람 이름 [cite: 861]
receiver_phone_number VARCHAR(20) NOT NULL, -- 받는사람 전화번호 [cite: 862]
ribbon_sender TEXT, -- 리본문구 보내는사람 [cite: 863]
ribbon_congratulations TEXT, -- 리본문구 경조사어 [cite: 864]
card_message TEXT, -- 카드메세지 [cite: 865]
rpa_status VARCHAR(20) DEFAULT 'ready', -- RPA 처리 상태 ('ready', 'success', 'fail')
created_at TIMESTAMPTZ DEFAULT NOW(), -- [cite: 866]
```

FOREIGN KEY (call\_history\_id) REFERENCES server\_call\_history(id) ON DELETE CASCADE

);

```

CREATE TABLE setting_info ( -- 신규 생성해야 함
  id SERIAL PRIMARY KEY,
  shop_key INT NOT NULL UNIQUE,          -- 꽃가게 고유 KEY (회원정보와 1:1 매핑)

  -- [알림 시스템 필드 설정]
  use_notification CHAR(1) DEFAULT 'Y',    -- 알림톡/문자 발송 여부 ('Y' 또는 'N')
  notification_phone_number VARCHAR(20) DEFAULT NULL, -- ★ 알림을 수신할 사장님 핸드폰 번호 (국번 포함)
  rpa_success_message TEXT DEFAULT '{channel} 주문 {count}건 꽃가게 관리 프로그램에 입력 완료했습니다.', -- 성공 보고 문구
  rpa_fail_message TEXT DEFAULT '[ggotAI 경고] {channel} 주문 자동 입력 실패! 수동 확인 바랍니다.', -- 실패 경고 문구

  -- [기기 및 채널 설정]
  order_hp_1 VARCHAR(20) NOT NULL,          -- 주문핸드폰1
  order_hp_2 VARCHAR(20) DEFAULT NULL,      -- 주문핸드폰2
  order_landline_1 VARCHAR(20) DEFAULT NULL, -- 주문일반전화1
  order_landline_2 VARCHAR(20) DEFAULT NULL, -- 주문일반전화2

  -- [외부 연동 설정]
  shopping_mall_url VARCHAR(255) DEFAULT NULL, -- 쇼핑몰 관리자주소
  shopping_mall_id VARCHAR(50) DEFAULT NULL,   -- 쇼핑몰 아이디
  shopping_mall_password VARCHAR(255) DEFAULT NULL, -- 쇼핑몰 패스워드 (암호화 저장)
  intranet_url VARCHAR(255) DEFAULT NULL,      -- 인트라넷주소
  intranet_id VARCHAR(50) DEFAULT NULL,        -- 인트라넷 아이디
  intranet_password VARCHAR(255) DEFAULT NULL, -- 인트라넷 패스워드 (암호화 저장)

  shopping_mall_check_interval INT DEFAULT 10; -- 쇼핑몰 확인 간격 (분 단위) [cite: 3559]
  intranet_check_interval INT DEFAULT 30;      -- 인트라넷 확인 간격 (분 단위) [cite: 3561]

  created_at TIMESTAMP DEFAULT NOW()
);

```

```

CREATE TABLE member_info ( -- 기존 Supabase의 member_info Table 수정해야 됨
  id SERIAL PRIMARY KEY,          -- 고유 KEY값 (이 값이 다른 테이블의 shop_key가 됨)
  username VARCHAR(50) NOT NULL UNIQUE, -- 아이디
  password VARCHAR(255) NOT NULL,      -- 비밀번호
  shop_name VARCHAR(50) NOT NULL,      -- 꽃집이름
  representative_name VARCHAR(50) NOT NULL, -- 대표자
  landline_number VARCHAR(20) DEFAULT NULL, -- 가게전화
  mobile_number VARCHAR(20) NOT NULL,   -- 대표 핸드폰 번호
  email VARCHAR(100) DEFAULT NULL,      -- 이메일
  address VARCHAR(255) DEFAULT NULL,    -- 주소
  address_detail VARCHAR(50) DEFAULT NULL, -- 상세주소
  is_approved CHAR(1) DEFAULT 'N',      -- 회원가입 승인 여부 ('Y' 또는 'N')
  created_at TIMESTAMP DEFAULT NOW()
);

```

## II. ggotAlorder

(B.실시간 주문 수집 및 입력(PC 백그라운드 서비스))

# 1. ggotAlorder(B.실시간 주문 수집 및 입력(PC 백그라운드 서비스))

## (1) 실행

ggotAlorder는 윈도우 OS가 켜질 때 자동으로 시작되는 독립적인 '윈도우 백그라운드 서비스'

## (2) 작동중임을 확인하는 아이콘

- 'ggotAlorder' 실행 → 윈도우 시계 옆(트레이)에 가벼운 'ggotAI 아이콘' 상주 (작동 중임을 알리는 용도)
- 사장님이 그 아이콘을 [더블 클릭] 하면, 'ggotAlya' 를 자동으로 가동하면서 '상황판 탭'을 바로 열어줌.

(3) '주문수집중단' 및 '주문수집시작'은 "C. 관리용 UI 애플리케이션(PC 포그라운드 액티비티) ggotAlya"의 상황판에서 제어함.  
기본적으로는 ggotAlorder가 실행되면 주문수집가능 상태로 됨

아래 지시사항과 첨부된 ggotAlorder 설계 문서를 바탕으로, Python 기반의 PC 백그라운드 주문 수집 서비스인 'ggotAlorder'의 전체 코어 엔진 아키텍처와 패키지 구조를 설계하고 핵심 코드를 작성해줘.

## 1. 프로세스 생명주기 및 윈도우 서비스 (7~8페이지):

- 본 프로그램은 Windows Service 형태로 백그라운드에서 독립 구동되며, 윈도우 트레이 아이콘으로 상주해. 자체 종료는 없고 포그라운드 UI(ggotAlya)의 명령(net stop ggotAlorder)으로 제어되는 구조의 뼈대를 만들어줘. 트레이 아이콘 더블클릭 시 ggotAlya를 가동하는 통신 핸들러를 포함해줘.

## 2. 수신 채널별 데이터 인제션 연동 (11~13페이지):

- [가게전화]: FastAPI 기반 엔드포인트(/api/v1/gate-phone/upload)를 열어, 통신사 서버로부터 Webhook(POST Multipart)으로 넘어오는 음성 파일과 발신자 정보를 비동기로 수신하는 코드를 작성해줘.
- [핸드폰/가게음성]: supabase-py 라이브러리를 활용해 'server\_call\_history' 테이블의 INSERT 이벤트를 Realtime(웹소켓)으로 24시간 감시하다가, 신규 데이터 유입 즉시 'on\_new\_call\_received' 콜백을 구동하는 구조를 빌드해줘.

## 3. 파이프라인 이원화 및 AI 분석 엔진 (9, 17~18페이지):

- 음성 채널 수신 시, faster-whisper 모델을 연동하여 STT를 수행하고, 추출된 비정형 텍스트를 Gemini API에 주입하여 11가지 정형화 양식(JSON)을 추출해줘. 11가지 항목 중 공백이 3개 이상이면 꽃 추문이 아닌 것으로 판단(is\_order='N')하여 서버 임시 음성 파일을 삭제하고, 성공 시에는 order\_details에 적재해줘.
- 텍스트 채널(쇼핑몰/인터넷) 수신 시에는 Whisper/Gemini 단계를 완전히 패스하고 즉시 order\_details에 rpa\_status='ready'로 다이렉트 적재하는 분기 로직을 구현해줘.

## 4. 인터넷 크롤링 및 RPA 동기화 큐 (15~16, 18페이지):

- Playwright 라이브러리를 활용해 setting\_info의 체크 간격 주기에 따라 headless 모드로 자동 로그인 및 목록 스크래핑을 수행하는 스케줄러 뼈대를 작성해줘.
- 다중 채널에서 동시다발적으로 유입되는 주문을 처리할 때 윈도우 제어가 꼬이지 않도록 'asyncio.Lock()'을 진입점에 걸어 순차적으로 실행되는 싱글턴 RPA 호출 큐를 구현해줘. 만약 PC 내에 꽃집 관리 프로그램 창이 활성화되어 있지 않다면 자동으로 엑셀 파일 및 인수증을 생성하여 디스크에 보존하는 예외 로직을 포함해줘.

## 5. 결과 보고 알림록 (6, 8페이지):

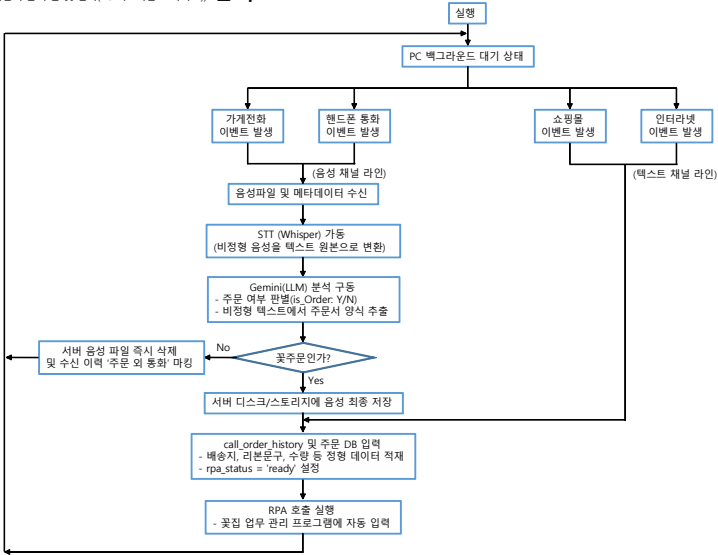
- RPA 수행 완료 후, setting\_info의 'use\_notification'이 'Y'이면 'notification\_phone\_number'로, 누락 시 member\_info의 'mobile\_number'로 알림 메시지를 전송해줘. 메시지 발송 시 원본 문구 내의 '{channel}'과 '{count}' 변수를 실시간 치환(Replace)하여 발송 모듈로 토스하는 파이프라인 정형화 함수를 작성해줘.

4개의 Table중에서 2개는 신규생성하고, 2개는 수정해야됨

1. server\_call\_history : 기존 Supabase의 Server\_call\_history Table 수정해야 됨
2. member\_info : 기존Supabase의 member\_info Table 수정해야 됨
3. order\_details: 신규 생성해야 함
4. setting\_info: 신규 생성해야 함



2. ggotAlorder (B.실시간 주문 수집 및 입력(PC 백그라운드 서비스)) 순서도



### 3. 웹(Back-end)의 채널별 데이터 수집 규격 (Data Ingestion)

1. [가게 전화 채널]: 물리적 PC-전화 연결을 배제하고, 인터넷전화(VoIP) CTI 인프라를 연동한다.  
통신사 서버가 녹음한 음성 파일과 발신자 정보를 백엔드 웹 API로 다이렉트 수신(WebHook)하도록 구현한다.
2. [핸드폰 앱 채널]: 단말 앱(ggotAlhp)이 통화 종료 후 서버(supabase)에 파일 및 통화음성파일을 적재하면,  
서버는 ggotAlorder에 실시간 신호를 전송하여 Whisper(STT) 및 Gemini(LLM) 파이프라인을 즉시 구동한다.
3. [쇼핑몰 채널]: 화면 크롤링 방식을 지양하고, 주문 접수 시 발송되는 이메일(SMTP) 알림 인터셉트 파싱 엔진 또는  
공식 주문 수집 API 연동 모듈을 구현하여 외부 사이트 변동에 따른 유지보수(A/S) 리스크를 최소화한다.
4. [인터넷 채널]: 화면 크롤링 방식으로 주문을 수집한다.
5. [가게 현장 음성 채널]: PC에 물리 마이크를 세팅하지 않는다.  
이미 개발된 스마트폰 앱(ggotAlhp) 내에 '현장 주문 녹음' 간이 UI를 추가하여, 사장님이 폰에 대고 말한 음성 파일을  
기존 핸드폰 통화 전송 API와 동일한 포맷으로 서버 DB에 적재 및 처리하도록 단일화한다.

### 3-1. [가게 전화 채널]: 물리적 PC-전화 연결을 배제하고, 인터넷전화(VoIP)-KT 인터넷 전화

음성 파일 업로드 완료(Webhook) 자체를 신호로 인식

대부분의 기업용 VoIP CTI 녹음 서비스는 **통화가 종료되는 즉시 녹음 파일을 생성**하도록 설계되어 있습니다.

따라서 백엔드 웹 서버 입장에서는 "통신사 녹음 서버로부터 음성 파일 업로드 요청(POST)이 들어온 시점"을 곧 통화 종료 시점으로 판단합니다.

#### 실제 작동 프로세스

1. 고객과 사장님이 일반전화(인터넷전화)로 통화를 합니다.
2. 두 사람이 전화를 끊으면, 통신사 교환기 시스템이 자동으로 통화 녹음을 마감하고 .wav 또는 .mp3 파일로 저장합니다.
3. 파일 저장에 끝나자마자 통신사 서버가 ggotAI 백엔드 API(/api/v1/gate-phone/upload)로 **음성 파일과 발신자 번호 정보를 담아 Multipart 업로드 요청**을 보냅니다.
4. 백엔드 웹은 이 파일 업로드 API가 호출되는 순간 "통화가 끝났으니 파일이 넘어왔구나"라고 인지하고, 즉시 다음 단계인 Whisper(STT)와 Gemini(LLM) 라인을 실행합니다.

#### 📞 바이브 코딩 시 AI에게 주입할 백엔드(FastAPI) 개념

바이브 코딩으로 백엔드 수신 코드를 짤 때, AI가 통신사 규격에 맞게 이벤트를 처리하도록 아래와 같은 구조를 제시하면 아주 좋습니다.

📞 바이브 코딩 시 AI에게 주입할 백엔드(FastAPI) 개념

바이브 코딩으로 백엔드 수신 코드를 짤 때, AI가 통신사 규격에 맞게 이벤트를 처리하도록 아래와 같은 구조를 제시하면 아주 좋습니다.

Python

```
from fastapi import FastAPI, File, UploadFile, Form
```

```
app = FastAPI()
```

# 통신사 서버가 통화 종료 후 음성 파일과 통화 정보를 따주는 엔드포인트

```
@app.post("/api/v1/gate-phone/upload")
```

```
async def receive_gate_phone_call(
```

```
    file: UploadFile = File(...),           # 통신사가 보낸 녹음 파일
    caller_number: str = Form(...),         # 전화를 건 고객 번호
    call_duration: int = Form(...),         # 통화 시간(초)
    user_phone_number: str = Form(...)       # 전화를 받은 가게 전화번호 [cite: 434]
```

```
):
```

```
    # 1. 이 API가 호출되었다는 것 자체가 "통화 종료"를 의미함
    print(f"📞 가게전화 통화종료 수신: {caller_number} → {user_phone_number} ({call_duration}초)")
```

```
    # 2. 파일 저장 후 곧바로 Whisper + Gemini 가동 코드로 연동
```

```
    # text = run_whisper(file)
    # is_order = run_Gemini(text)
```

```
    return {"status": "success", "message": "통화 종료 데이터 접수 완료"}
```

#### 💡 요약

하드웨어(PC-전화선) 연결 방식에서는 엡처럼 폰 내부 라이브러리로 감김을 감지해야 했지만, VoIP CTI 인프라 방식에서는 통신사가 통화가 끝나는 순간 "전화 끊겼다!" 하고 데이터(웹훅)나 녹음 파일을 백엔드로 직접 던져주기 때문에 백엔드가 실시간으로 즉시 알 수 있습니다.

### 3-2. [핸드폰 앱 채널]: 서버 Database에 주문음성파일 추가시 ggotAlorder가 파악하는 방법

Supabase 환경에서 'ggotAlorder'(PC 백그라운드 서비스)가 신규 주문 음성 파일 접수를 실시간으로 감지하여 Whisper와 Gemini를 구동하게 만드는 가장 핵심적인 방법(Realtime 기능 활용 및 Webhook 활용)을 알려드립니다.

방법 1. Supabase Realtime 기능 활용 (가장 추천, 웹소켓 방식)

Supabase는 테이블에 새로운 데이터가 추가(INSERT)되면, 이를 실시간 웹소켓(WebSocket) 채널을 통해 클라이언트(웹1 프로그램: "PC 백그라운드 서비스, ggotAlorder")에 즉시 쏘주는 Realtime 기능을 제공합니다.

이 방식을 쓰면 별도의 백엔드 알림 서버를 구축할 필요 없이, 'ggotAlorder'(PC 백그라운드 서비스) 프로그램 내부 코드 몇 줄만으로 실시간 수신 대기 상태를 만들 수 있습니다.

☞ 구현 메커니즘

핸드폰 앱(ggotAlhp)이 통화를 마치고 음성 파일을 업로드하면서 서버 DB의 server\_call\_history 테이블에 새로운 행을 추가(INSERT)합니다.

상시 켜져 있는 'ggotAlorder'(PC 백그라운드 서비스) 프로그램이 Supabase Realtime 채널을 구독(Subscribe)하고 대기하다가, 해당 테이블에 데이터가 들어오는 순간 즉시 알림(이벤트)을 수신합니다.

알림에 포함된 audio\_file\_name과 데이터 고유 ID를 바탕으로, 'ggotAlorder' 내부에서 Whisper와 Gemini 함수를 즉각 가동합니다.

가게 PC 내부에서 가볍게 돌아가는 백그라운드 프로그램을 만드실 때는 [방법 1 (Supabase Realtime 방식)]이 압도적으로 유리합니다.

• **이유:** 외부 웹hook 방식을 쓰려면 가게 PC에 외부 인터넷 신호가 들어올 수 있도록 공유기 포트포워딩 설정을 하거나 고정 IP 세팅을 해야 하는 번거로움이 있습니다.

• 반면 **Realtime 웹소켓 방식**은 웹1 프로그램(PC 백그라운드 서비스, ggotAlorder)이 Supabase 서버에 먼저 뿔대를 꽂고 대기하는 방식이라, 공유기 설정이나 IP 상태와 무관하게 24시간 안정적으로 신호를 받아들일 수 있습니다.

바이브 코딩 시 AI(Cursor/Claude)에게 프롬프트를 던질 때 이렇게 명시해 보세요:

"ggotAlorder' 프로그램은 파이썬으로 구동되는 백그라운드 서비스야. 데이터베이스는 Supabase를 사용해. Supabase의 supabase-py 라이브러리의 Realtime 채널 구독 기능을 사용해서,

server\_call\_history 테이블에 신규 INSERT가 일어나는 즉시 데이터를 패치해서 Whisper STT 함수를 호출하는 비동기 이벤트를 작성해줘."

이렇게 지시하시면 AI가 Supabase 환경에 완벽히 최적화된 실시간 수집 코드를 깔끔하게 작성해 줄 것입니다.

스름 바이트 코딩용 Python 코드 예시 ('ggotAlorder'가 감시하는 코드)  
'ggotAlorder' 프로그램을 Python 기반으로 개발하실 경우, 시에게 아래 뼈대 코드를 주고 고도화해달라고 하시면 됩니다.

```
Python
import asyncio
from supabase import create_client, Client
import json

SUPABASE_URL = "YOUR_SUPABASE_URL"
SUPABASE_KEY = "YOUR_SUPABASE_ANON_KEY"

supabase: Client = create_client(SUPABASE_URL, SUPABASE_KEY)

# 신규 주문 데이터가 들어왔을 때 실행될 실시간 콜백 함수
def on_new_call_received(payload):
    print("📞 신규 통화 데이터 수신 감지!")
    new_data = payload.get('record', {})

    call_id = new_data.get('id')
    audio_file = new_data.get('audio_file_name')
    user_phone = new_data.get('user_phone_number')

    print(f"🔍 분석 시작 -> ID: {call_id}, 파일명: {audio_file}, 발신단말: {user_phone}")

    # =====
    # 여기에 Whisper STT 및 Gemini(LLM) 구동 코드가 배치됩니다.
    # 예: text = run_whisper(audio_file)
    # is_order = run_Gemini(text)
    # =====

# Supabase Realtime 채널 구독 설정
def start_realtime_listener():
    # server_call_history 테이블의 INSERT 이벤트만 실시간 감시
    channel = supabase.channel("call_watcher")
    channel.on(
        "postgres_changes",
        {'event': "INSERT", "schema": "public", "table": "server_call_history"},
        on_new_call_received
    ).subscribe()

    print("🔄 웹1 백그라운드 서비스가 Supabase Realtime 감시를 시작했습니다...")
    # 백그라운드 서비스가 종료되지 않도록 무한 대기 루프 구동
    loop = asyncio.get_event_loop()
    loop.run_forever()

if __name__ == "__main__":
    start_realtime_listener()
    ⚠ 주의 사항 (Supabase 설정 필수): Supabase 대시보드 메뉴에서 Database -> Replication으로 이동한 후, server_call_history 테이블의 Realtime 기능(Source 테이블 활성화)을 반드시 켜주셔야 위 코드가 작동합니다.
```

### 3-3. [핸드폰 앱 채널]: 서버 Database에 주문음성파일 추가시 ggotAlorder가 파악하는 방법

가장 안정적인 업그레이드 방식은 각 쇼핑몰이 제공하는 공식 **주문 수집 API** 또는  
주문 발생 시 사장님 메일/문자로 날아오는 '주문 접수 알림 메일 파싱(Email Parsing) 시스템'을 도입하는 것입니다.  
타사 웹사이트 화면 변화에 영향을 받지 않아 A/S 발생률이 95% 이상 감소합니다.

### 3-4. [인터넷 채널]: "인터넷 크롤링 주문 수집 가이드 및 상세

바이브 코딩(Vibe Coding) 시 AI가 올바른 크롤링 봇과 스케줄러 코드를 짜도록 주입할 "인터넷 크롤링 주문 수집 가이드 및 상세 절차"입니다

[상세 명세] 인터넷 크롤링 주문 데이터 수집 절차  
인터넷 채널의 크롤링 수집은 웹1(PC 백그라운드 서비스, ggotAlorder) 내부의 독립적인 스케줄러 모듈로 구동되며, 총 5단계의 절차를 거칩니다.

#### 1단계: 크롤러 환경 초기화 및 자동 로그인 (Session 획득)

크롤러가 사람이 브라우저를 켜서 들어가는 것처럼 안전하게 인터넷에 접속하는 단계입니다.

headless 브라우저 구동: 서버 메모리 자원 절약을 위해 화면이 뜨지 않는 Headless 모드의 크롤러 라이브러리(Python의 Playwright 또는 Selenium)를 실행합니다.

보안 세팅 (User-Agent 변조): 인터넷 서버가 AI 봇의 접근을 차단하지 않도록, 일반 크롬 브라우저를 쓰는 것처럼 헤더 정보(User-Agent)를 사람의 값으로 변조하여 접속합니다.

자동 로그인 처리: 중앙 DB 서버에서 암호화되어 관리되고 있는 인터넷 주소, 아이디, 비밀번호를 가져와 복호화한 후, ID/PW 입력 필드의 CSS 선택자(Selector)를 찾아 값을 주입하고 로그인 버튼 클릭 이벤트를 발생시킵니다.

세션(Cookie) 유지: 매번 로그인하는 번거로움과 서버 부하를 줄이기 위해, 로그인 후 발급받은 세션 쿠키를 메모리에 유지합니다.

#### 2단계: 신규 주문 목록 페이지 타겟팅 및 주기적 감시 (Polling)

실시간 주문 수집을 위해 주문 목록 페이지를 주기적으로 확인하는 단계입니다.

주기적 호출 설정 (스케줄러): 웹1(PC 백그라운드 서비스, ggotAlorder) 내부에 백그라운드 타이머(예: APScheduler 라이브러리)를 가동하여 5분~10분 주기로 인터넷의 '신규 주문 내역' 또는 '접수 대기' 페이지 URL로 다이렉트 이동합니다.

HTML 돔(DOM) 구조 분석: 페이지가 로딩 완료되면 HTML 소스를 가져와 주문 테이블의 행(Row) 요소를 파싱합니다.

기존 주문 교차 검증 (중복 수집 방지): \* 크롤러가 읽어온 목록의 '인터넷 주문번호' 또는 '시간+고객명' 조합을 추출합니다.

중앙 DB의 server\_call\_history 및 주문 테이블을 조회하여 이미 수집된 데이터인지 대조합니다.

이미 존재하는 데이터라면 스킵(Skip)하고, DB에 없는 고유한 주문 번호가 발견되면 '신규 주문'으로 인식하고 다음 단계로 이동합니다.

#### 3단계: 상세 데이터 추출 및 데이터 정형화

목록 화면에서는 누락될 수 있는 상세 리본 문구, 배송지 주소 등을 정확히 읽어오는 단계입니다.

상세 페이지 진입: 신규 주문의 '상세보기' 버튼 스크립트를 클릭하거나 상세 페이지 URL로 브라우저를 이동시킵니다.

텍스트 긁어오기 (Scraping): 상세 화면 내에서 RPA와 Gemini가 필요로 하는 아래 항목들의 표(Table) 위치나 텍스트 태그를 정확히 정밀 퀴리하여 데이터 객체(JSON)로 변환합니다.

추출 항목: 상품명, 수량, 배송지 주소, 받는 분 성함/연락처, 보내는 분 성함, 배송 희망 일시, 리본 및 카드 문구

임시 이미지 처리 (선택사항): 만약 배송할 꽃의 참고 이미지나 특이사항이 이미지 형태로 첨부되어 있다면, 해당 이미지 URL을 다운로드하여 Supabase Storage(스토리지)에 업로드하고 주소 값을 맵핑합니다.

#### 4단계: 중앙 데이터베이스 적재 (Data Insertion)

크롤링한 비정형/반정형 데이터를 우리 시스템 아키텍처에 맞게 안전하게 집어넣는 단계입니다.

수집 이력 생성: `server_call_history` 테이블에 데이터를 밀어 넣습니다.

이때 `user_phone_number`는 인터넷 수집 기기 계정 코드로 마킹하고, 음성이 없는 데이터이므로 `audio_file_name`은 'INTRANET\_CRAWLED'로 대체합니다.

크롤링으로 이미 텍스트가 추출되었으므로 `stt_text` 필드에 들어온 텍스트 원본을 바로 저장합니다.

주문 상세 데이터 즉시 적재: 들어온 데이터가 이미 정형화 구조를 갖추고 있으므로, 굳이 Gemini(LLM)를 거치지 않고 곧바로 `order_details` 테이블에 정형화된 상태로 값을 INSERT합니다. 이때 RPA 상태 필드인 `rpa_status`는 'READY'(대기) 상태로 설정합니다.

#### 5단계: 예외 처리 및 알림 (A/S 대응 핵심 프로세스)

인터넷 사이트 변경으로 크롤러가 터졌을 때 즉시 대응하기 위한 가장 중요한 보안 단계입니다.

구조 변경 감지 (DOM Exception): 인터넷 사이트가 리뉴얼되어 로그인 버튼 위치가 바뀌거나 주문 테이블 태그가 바뀌면 크롤러는 HTML 요소를 찾지 못해 예러 (ElementNotFoundException)를 뱉습니다.

유형 대기 방지: 예러가 발생하면 무한정 대기하거나 프로그램이 다운되지 않도록 즉시 해당 회차의 크롤링 루프를 탈출(try-except-finally)시킵니다.

관리자 SOS 알림톡 발송: 크롤링 실패 횟수가 연속 3회 이상 누적되면, 인터넷 구조 변동으로 판단하고 사장님의 카카오톡(웹2 "관리용 UI 애플리케이션 ggotAly"에서 설정한 설정 카톡 ID)으로 "[ggotAI 경고] 인터넷 사이트 구조 변경 감지! 크롤러 정지 상태이므로 수동 확인 바람!"이라는 예러 메시지를 자동 발송하도록 설계합니다.

#### 🔥 바이브 코딩용 AI 명령 프롬프트 팁

인터넷 크롤링 엔진 개발을 시작할 때 AI에게 아래와 같이 구체적인 프롬프트를 던지시면 원 원하시는 절차가 100% 반영된 코드를 얻을 수 있습니다.

"웹1(PC 백그라운드 서비스, ggotAIorder) 프로그램 내에 인터넷 주문 수집을 위한 Playwright 기반의 Python 크롤러 모듈을 만들어줘.

`member_info` 테이블에서 인터넷 계정 정보를 가져와서 Headless 브라우저로 자동 로그인을 수행해야 해.

10분 주기로 주문 목록을 폴링하되, 데이터가 수집되면 Supabase DB에 기존 주문 번호가 있는지 체크해서 중복 수집을 차단해줘.

신규 주문 발견 시 상세 페이지의 상품명, 주소, 리본 문구를 읽어와서 `order_details` 테이블에 `rpa_status='READY'`로 인서트 해야 해.

만약 인터넷 HTML 태그를 찾지 못하는 크롤링 예러가 발생하면, 즉시 예외 처리를 하고 사장님 카카오톡 API로 알림을 쓰는 안전장치 코드를 포함해줘."



#### 4. STT (Whisper)

##### 1. 변환 프로세스



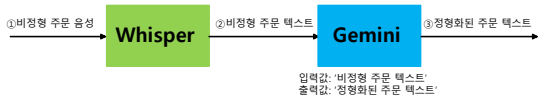
입력값: '비정형 주문 음성'  
출력값: '비정형 주문 텍스트'

- ①비정형 주문 음성: 고객이 핸드폰 또는 가게 전화로 꽃 주문을 한 통화 음성으로서, 꽃가게 관리 프로그램에 입력할 수 있는 양식, 또는 주문 양식에 맞게 조정되지 않은 데이터
- ②비정형 주문 텍스트: 통화내용 텍스트로서, 꽃가게 관리 프로그램에 입력할 수 있는 양식, 또는 주문 양식에 맞게 조정되지 않은 데이터

## 5. Gemini(LLM) 분석

- Gemini(LLM) 분석 구동
- 주문 여부 판별(is\_Order: Y/N)
  - 비정형 텍스트에서 주문서 양식 추출

### 1. 분석 프로세스



- ① 비정형 주문 음성: 고객이 핸드폰 또는 가게 전화로 꽃 주문을 한 통화 음성으로서, 꽃가게 관리 프로그램에 입력할 수 있는 양식, 또는 주문 양식에 맞게 조정되지 않은 데이터
- ② 비정형 주문 텍스트: 통화내용 텍스트로서, 꽃가게 관리 프로그램에 입력할 수 있는 양식, 또는 주문 양식에 맞게 조정되지 않은 데이터
- ③ 정형화된 주문 음성: 꽃가게 관리 프로그램에 입력하거나, 특정 주문 양식에 맞게 조정된 주문 텍스트

### 2. 분석 내용(정형화된 텍스트)

#### (1) 통화 내용을 주문서 양식에 맞게 정형화된 텍스트로 정리

- ① 주문고객:
- ② 주문고객 연락처:
- ③ 상품명:
- ④ 상품가격:
- ⑤ 상품수량:
- ⑥ 배달장소:
- ⑦ 받는사람:
- ⑧ 받는사람 연락처:
- ⑨ 리본문구(경조사어):
- ⑩ 리본문구(보내는 사람):
- ⑪ 카드메세지:

로 분리해주세요. 내용이 없는 항목은 비워두세요.

#### (2) 통화 내용이 꽃 주문 내용인지 판단

- 정형화된 텍스트 내용의 3개 이상 항목이 공백이면, 꽃 주문이 아닌걸로 판단함.
- "Gemini가 리턴한 JSON 구조체에서 11가지 주문서 항목 중 값이 비어있는 항목(None 또는 공백)이 3개 이상일 경우, 시스템은 자동으로 'is\_order = N'으로 판별하고 설계서 9페이지 순서도에 따라 서버 임시 파일을 즉시 삭제하는 예외 처리 파이션 함수를 작성해줘."

## 4. RPA 처리

"RPA 작업은 무조건 하나의 큐(Queue)를 통해 순차적으로 실행되어야 하며, 하나의 RPA가 실행 중일 때는 다른 RPA 작업이 대기하도록, 싱글턴(Singleton) 또는 락(Lock) 메커니즘을 구현해줘"

"PC 내에서 여러 채널(전화, 쇼핑물, 인터넷)의 주문이 동시다발적으로 입력 완료되어 RPA 호출이 겹치더라도, 윈도우 화면 제어가 꼬이지 않도록 파이썬의 threading.Lock() 또는 비동기 \*\*asyncio.Lock()\*\*을 사용하여 RPA 진입점을 완전 동기화 제어해줘."

"RPA 모듈 구동 시, 윈도우 상에서 꽃가게 관리 프로그램 창(Window Title)을 찾지 못할 경우 프로세스를 터뜨리지 말고, 수집된 정형 데이터를 바탕으로 지정된 폴더에 엑셀(.xlsx) 파일과 텍스트 형태의 인수증 영수증 파일을 자동 생성하여 저장하도록 예외 분기를 짜줘."